WATERMARK FORGERY ATTACK TRUSTWORTHY MACHINE LEARNING

1 INTRODUCTION

In this assignment, the goal is to perform a watermark forgery attack under almost black-box conditions. We are given 200 clean target images and 8 groups of watermarked source images, but we do not know which watermarking algorithm is used. The task is to transfer the hidden watermark from each source group to the corresponding clean target images while keeping the image quality as high as possible. Since the final score depends on both watermark detection and image quality, my goal was to find a good balance between these two.

2 APPROACH

My final solution uses a hybrid approach because I found that one method does not work well for all watermark groups.

For WM_7, after analysing all source images, I found that every image contains the same TrustMark message. Instead of estimating the watermark, I directly encoded this recovered message into every clean target image using the TrustMark encoder. This gives very good watermark recovery and also keeps the LPIPS value very low.

For the remaining watermark groups (WM_1 to WM_6 and WM_8), direct encoding was not possible. So, I used an iterative Non-Local Means (NLM) based watermark extraction method.

First, I applied NLM denoising to all source images of one watermark group. Then I calculated the residual between the original image and the denoised image. By averaging these residuals, I got the common watermark signal shared by all images.

To improve the watermark estimation, I repeated this process for eight iterations. In every iteration, the previously estimated watermark was removed before denoising again, which helped to extract a cleaner watermark signal.

Apart from the average watermark signal, I also extracted an individual residual for every image and another high-frequency signal using Gaussian filtering. These three signals were combined together to generate the final watermark signal.

$$S = w_1 S_{avg} + w_2 S_{individual} + w_3 S_{gaussian}$$

where

$$w_1 = 1.0, \qquad w_2 = 0.5, \qquad w_3 = 0.5$$

If the watermark is added too strongly, the LPIPS value increases. So, I used binary search to automatically find the best scaling factor for every image. This helped me keep the LPIPS close to the target value while still preserving as much watermark information as possible.

Image quality was measured using LPIPS with the AlexNet backbone.

The main implementation settings are:

Method	Hybrid TrustMark + Iterative NLM
TrustMark Group	WM_7
Other Groups	WM_1–WM_6, WM_8
NLM Iterations	8
Template Window	7
Search Window	21
Gaussian Sigma	1.0
LPIPS Network	AlexNet
Scaling	Binary Search
Output	ZIP file with 200 PNG images

3 PUBLIC LEADERBOARD RESULT

For the final submission, I generated a ZIP file containing all 200 forged images in the required format. The hybrid approach uses direct TrustMark encoding for WM_7 and iterative NLM watermark estimation for the remaining watermark groups. This combination gives a good balance between watermark detection accuracy and visual quality.

My best public leaderboard result was:

```
Public leaderboard score:  0.605296
```

This score was obtained by submitting the generated ZIP file to the evaluation server.

4 CONCLUSION

From this assignment, I understood that invisible watermarking systems are not fully secure. Even if we do not know how the watermarking algorithm works, it is still possible to analyse watermarked images, recover the hidden watermark, and apply it to other images.

This can become a serious problem in real-world applications. Someone can create fake ownership proofs, copy watermarks to different images, or misuse watermarking systems to make fake content look authentic. Because of this, watermarking methods should not only focus on embedding watermarks but also on protecting them from copying and forgery attacks.

Overall, the hybrid approach worked well because it combines direct TrustMark encoding with iterative NLM watermark extraction. It gives good watermark recovery while keeping the visual quality high, which helps achieve a better overall score.

GitHub Repository: `https://github.com/Jagadeeswar-reddy-c/TML26_X_04`